

TP3: Relatório de Progresso e Validação de Modelos

1. Estratégia de Modelagem

Neste estágio do projeto, estamos focando em **Análise de Sentimento Multi-classe** (Positivo, Negativo, Neutro) aplicada aos comentários de chat do YouTube.

As técnicas sendo testadas incluem:

- **Zero-Shot Learning com LLMs (vLLM)**: Utilizando modelos de linguagem de larga escala para gerar labels automáticos e explicações.
- **Fine-tuning de BERT (BERTimbau)**: Treinamento supervisionado utilizando uma base de dados rotulada manualmente para capturar nuances do futebol brasileiro.
- **Métricas de Engajamento**: Integração de sentimento com momentum da partida (xT).

```
In [1]: import pandas as pd
import numpy as np
from sklearn.metrics import cohen_kappa_score, classification_report, accuracy_score
import matplotlib.pyplot as plt
import seaborn as sns

# Configurações de visualização
sns.set_theme(style="whitegrid")
```

2. Carregamento e Preparação dos Dados Rotulados

Cada exemplo foi rotulado por dois anotadores diferentes entre um grupo de 5 (Igor, Ivan, Vitor, Alvaro, Dani).

```
In [2]: file_path = '../data/processed/consolidated/manual_labeling_bundesliga_LABELS.xlsx'
xl = pd.ExcelFile(file_path)
raters = ['Igor', 'Ivan', 'Vitor', 'Alvaro', 'Dani']

def map_sentiment(s):
    if not isinstance(s, str): return np.nan
    s = s.strip().lower()
    if 'negativo' in s: return 0
    if 'neutro' in s: return 1
    if 'positivo' in s: return 2
    return np.nan

# Carregar todas as abas
dfs = {}
for rater in raters:
    df = xl.parse(rater)
```

```

df['sentimento_num'] = df['sentimento'].apply(map_sentiment)
dfs[rater] = df[['comment_id', 'sentimento_num']].dropna()

# Cruzar labels para calcular Kappa
pairs = []
master_df = xl.parse('Master')
for idx, row in master_df.iterrows():
    cid = row['comment_id']
    r_list = [r.strip() for r in str(row['pair']).split(',')]
    if len(r_list) == 2:
        r1, r2 = r_list
        try:
            val1 = dfs[r1][dfs[r1]['comment_id'] == cid]['sentimento_num'].v
            val2 = dfs[r2][dfs[r2]['comment_id'] == cid]['sentimento_num'].v
            pairs.append({'comment_id': cid, 'rater1': r1, 'rater2': r2, 'la
        except:
            continue

pairs_df = pd.DataFrame(pairs)
print(f"Total de pares recuperados: {len(pairs_df)}")

```

Total de pares recuperados: 999

3. Inter-Rater Reliability (Cohen's Kappa)

O Kappa de Cohen mede a concordância entre dois avaliadores, descontando a concordância ao acaso.

```

In [3]: kappa = cohen_kappa_score(pairs_df['label1'], pairs_df['label2'])
print(f"Cohen's Kappa Global: {kappa:.3f}")

def interpret_kappa(k):
    if k < 0: return "Sem concordância"
    if k <= 0.20: return "Leve"
    if k <= 0.40: return "Razoável"
    if k <= 0.60: return "Moderada"
    if k <= 0.80: return "Substancial"
    return "Quase Perfeita"

print(f"Interpretação: {interpret_kappa(kappa)}")

```

Cohen's Kappa Global: 0.472

Interpretação: Moderada

4. Criação do Golden Set

O Golden Set será definido pela concordância entre os dois anotadores. Em caso de divergência, marcaremos para revisão ou usaremos um critério de desempate (ex: priorizar labels não-neutros ou média).

```

In [4]: # Golden set: Apenas onde houve concordância absoluta para garantir qualidade
golden_df = pairs_df[pairs_df['label1'] == pairs_df['label2']].copy()

```

```
golden_df['final_label'] = golden_df['label1']

# Adicionar a mensagem original para avaliação
full_master = xl.parse('Master')
golden_df = golden_df.merge(full_master[['comment_id', 'mensagem']], on='comment_id')

print(f"Tamanho do Golden Set (concordância total): {len(golden_df)}")
```

Tamanho do Golden Set (concordância total): 652

5. Avaliação do Modelo vLLM

Vamos comparar as predições do modelo vLLM com o nosso Golden Set humano.

```
In [5]: vllm_df = pd.read_csv('../data/processed/consolidated/gold_standard_vllm.csv')
# Nota: O vLLM exportado já tem comment_id sequencial ou mensagem correspondente
# Vamos cruzar por mensagem para garantir o match correto.
vllm_df['mensagem_clean'] = vllm_df['mensagem'].str.strip()
golden_df['mensagem_clean'] = golden_df['mensagem'].str.strip()

eval_df = golden_df.merge(vllm_df[['mensagem_clean', 'sentiment_vllm']], on='mensagem_clean')

print(f"Exemplos para avaliação: {len(eval_df)}")

y_true = eval_df['final_label']
y_pred = eval_df['sentiment_vllm']

print("\n--- Métricas Iniciais (vLLM vs Humano) ---")
print(f"Accuracy: {accuracy_score(y_true, y_pred):.3f}")
print(f"F1-Score (Weighted): {f1_score(y_true, y_pred, average='weighted'):.3f}")
print(f"Cohen's Kappa Score: {cohen_kappa_score(y_true, y_pred):.3f}")
print("\nRelatório de Classificação:")
print(classification_report(y_true, y_pred, target_names=['Negativo', 'Neutro', 'Positivo']))
```

Exemplos para avaliação: 716

--- Métricas Iniciais (vLLM vs Humano) ---

Accuracy: 0.690

F1-Score (Weighted): 0.674

Cohen's Kappa Score: 0.501

Relatório de Classificação:

	precision	recall	f1-score	support
Negativo	0.61	0.96	0.74	267
Neutro	0.81	0.60	0.69	314
Positivo	0.82	0.36	0.50	135
accuracy			0.69	716
macro avg	0.74	0.64	0.64	716
weighted avg	0.73	0.69	0.67	716

6. Avaliação do Modelo Otimizado (Llama 3.1 8B FP8)

Nesta etapa, otimizamos o pipeline utilizando o modelo `Llama-3.1-8B-Instruct-FP8` em conjunto com um prompt estruturado refinado (`prompt_v5`), instruindo o modelo a considerar regras estritas para a classe `Neutro` (default) e gírias de futebol e apostas (como *bagre*, *green*, *red*, *pipocar*).

```
In [6]: # Carregar predições do modelo otimizado
optimized_df = pd.read_csv('../data/processed/consolidated/llama_eval_result')
optimized_df['mensagem_clean'] = optimized_df['mensagem'].str.strip()

eval_opt_df = golden_df.merge(optimized_df[['mensagem_clean', 'pred']], on='mensagem_clean')

print(f"Exemplos para avaliação (Otimizado): {len(eval_opt_df)}")

y_true_opt = eval_opt_df['final_label']
y_pred_opt = eval_opt_df['pred']

print("\n--- Métricas Otimizado (Llama 8B FP8 vs Humano) ---")
print(f"Accuracy: {accuracy_score(y_true_opt, y_pred_opt):.3f}")
print(f"F1-Score (Weighted): {f1_score(y_true_opt, y_pred_opt, average='weighted'):.3f}")
print(f"Cohen's Kappa Score: {cohen_kappa_score(y_true_opt, y_pred_opt):.3f}")
print("\nRelatório de Classificação:")
print(classification_report(y_true_opt, y_pred_opt, target_names=['Negativo', 'Neutro', 'Positivo']))
```

Exemplos para avaliação (Otimizado): 712

```
--- Métricas Otimizado (Llama 8B FP8 vs Humano) ---
Accuracy: 0.805
F1-Score (Weighted): 0.806
Cohen's Kappa Score: 0.695
```

Relatório de Classificação:

	precision	recall	f1-score	support
Negativo	0.83	0.82	0.82	266
Neutro	0.84	0.80	0.82	311
Positivo	0.70	0.79	0.74	135
accuracy			0.80	712
macro avg	0.79	0.80	0.79	712
weighted avg	0.81	0.80	0.81	712

7. Avaliação do Modelo Destilado (BERTimbau)

Como próximo passo de nossa estratégia de modelagem, realizamos a destilação de conhecimento (Knowledge Distillation) treinando o modelo **BERTimbau** (`neuralmind/bert-base-portuguese-cased`) sobre o dataset de 2.000 mensagens rotuladas pelo modelo `Llama-3.1-8B-FP8`.

Abaixo, avaliamos a performance deste classificador leve e especializado contra o nosso Golden Set humano.

```
In [7]: # Carregar tokenizador e modelo BERTimbau finetunado
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch

model_path = '../bertimbau-bundesliga'
tokenizer = AutoTokenizer.from_pretrained(model_path)
model = AutoModelForSequenceClassification.from_pretrained(model_path)
device = "cuda" if torch.cuda.is_available() else "cpu"
model.to(device)
model.eval()

# Predição no eval_opt_df
messages_eval = eval_opt_df['mensagem'].tolist()
preds = []
batch_size = 64
for i in range(0, len(messages_eval), batch_size):
    batch = messages_eval[i:i+batch_size]
    inputs = tokenizer(batch, padding=True, truncation=True, max_length=128,
    inputs = {k: v.to(device) for k, v in inputs.items()}
    with torch.no_grad():
        outputs = model(**inputs)
        batch_preds = torch.argmax(outputs.logits, dim=-1).cpu().tolist()
        preds.extend(batch_preds)

eval_opt_df['pred_bertimbau'] = preds
y_pred_bert = eval_opt_df['pred_bertimbau']

print(f"Exemplos para avaliação (BERTimbau): {len(eval_opt_df)}")
print("\n--- Métricas BERTimbau vs Humano ---")
print(f"Accuracy: {accuracy_score(y_true_opt, y_pred_bert):.3f}")
print(f"F1-Score (Weighted): {f1_score(y_true_opt, y_pred_bert, average='wei")
print(f"Cohen's Kappa Score: {cohen_kappa_score(y_true_opt, y_pred_bert):.3f")
print("\nRelatório de Classificação:")
print(classification_report(y_true_opt, y_pred_bert, target_names=['Negativo
```

```
/home/igu/faculdade/cdaf/.venv/lib/python3.12/site-packages/tqdm/auto.py:21:
TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See h
https://ipywidgets.readthedocs.io/en/stable/user_install.html
```

```
from .autonotebook import tqdm as notebook_tqdm
```

```
Loading weights: 0%| | 0/201 [00:00<?, ?i
t/s]
```

```
Loading weights: 100%| | 201/201 [00:00<00:00, 10674.83i
t/s]
```

Exemplos para avaliação (BERTimbau): 712

--- Métricas BERTimbau vs Humano ---

Accuracy: 0.808

F1-Score (Weighted): 0.808

Cohen's Kappa Score: 0.698

Relatório de Classificação:

	precision	recall	f1-score	support
Negativo	0.84	0.83	0.83	266
Neutro	0.83	0.80	0.81	311
Positivo	0.71	0.78	0.74	135
accuracy			0.81	712
macro avg	0.79	0.80	0.80	712
weighted avg	0.81	0.81	0.81	712

8. Comparação de Métricas e Discussão sobre o Cohen's Kappa

Abaixo, consolidamos a comparação entre os anotadores humanos e as versões dos modelos:

Relação	Métrica de Concordância	Valor	Nível de Concordância
Humano vs Humano	Cohen's Kappa Global	0.472	Moderada
Baseline vLLM vs Humano	Cohen's Kappa / Acurácia	0.501 / 69.0%	Moderada
Llama-3.1-8B-FP8 (Otimizado) vs Humano	Cohen's Kappa / Acurácia	0.690 / 80.2%	Substancial
BERTimbau (Destilado) vs Humano	Cohen's Kappa / Acurácia	0.698 / 80.8%	Substancial

Discussão:

- Concordância Humana (0.472):** A concordância moderada entre os rotuladores humanos evidencia a alta subjetividade dos comentários do chat da CazéTV. Gírias de aposta misturadas com torcida criam divergências naturais de interpretação.
- Modelo Otimizado Llama 8B (0.690):** Ao refinar o prompt para mapear explicitamente os casos de sarcasmo, gírias e definir a classe Neutro como default, o modelo de 8B obteve uma concordância de **0.690** com os anotadores humanos (substancial).
- BERTimbau Destilado (0.698):** Treinar o modelo BERTimbau (110M parâmetros) na base de 2.000 comentários classificados pelo Llama 8B resultou em um Cohen's Kappa de **0.698** e acurácia de **80.8%**, ligeiramente superando o professor de 8B. Isso demonstra uma destilação de alta eficácia, permitindo usar um classificador extremamente veloz e compacto sem perda de acurácia.

9. Insights Preliminares

- Dificuldade em Sarcasmo:** Notamos que o vLLM às vezes interpreta gírias de aposta como negativas quando são celebrações.
- Concordância Humana:** O Kappa obtido indica o quão subjetiva é a tarefa, sugerindo a necessidade de diretrizes de anotação mais claras.
- Performance:** Resultados iniciais mostram que o modelo base é promissor, mas necessita de ajuste fino para o domínio de futebol.

10. Cronograma e Próximos Passos

Data	Atividade	Objetivo
05/Jun	Refinamento do Golden Set	Resolver divergências manuais nos ~10% restantes
10/Jun	Treinamento BERTimbau	Superar baseline do vLLM com modelo especializado
15/Jun	Tuning de Hiperparâmetros	Otimizar Learning Rate e Batch Size
20/Jun	Integração Final	Correlacionar sentimento com xT e Match Momentum
25/Jun	Entrega Final	Relatório consolidado e Dashboard

Métricas escolhidas: F1-Score (Weighted) é a principal, pois as classes são desbalanceadas (muito neutro).